

AWS Project Used in Production

Project Documentation by Praveen

Date: February 2026

Table of Contents

1. Project Overview
2. Architecture Diagram
3. Prerequisites
4. Step-by-Step Implementation
 - 4.1 VPC and Network Setup
 - 4.2 EC2 Instance Configuration
 - 4.3 SSH Connection using PuTTY
 - 4.4 Apache Installation and Configuration
 - 4.5 HTML Deployment
 - 4.6 Testing and Verification
5. Security Configuration
6. Troubleshooting

1. Project Overview

This project demonstrates the projects used production infrastructure on Amazon Web Services (AWS). The architecture implements best practices for cloud-based web hosting, including multiple availability zones, load balancing, and secure network design.

Project Objectives

- Deploy a scalable VPC infrastructure across multiple availability zones
- Configure EC2 instances to host Apache web server
- Implement secure SSH access using PuTTY
- Deploy custom HTML content on the web server
- Ensure high availability through load balancing and redundancy

Key Components

- VPC (Virtual Private Cloud): Custom network environment with CIDR block
- Public Subnets: Two subnets in different availability zones for high availability
- Private Subnets: Secure subnets for backend resources
- NAT Gateways: Enable private subnet instances to access the internet
- Application Load Balancer: Distributes traffic across multiple servers
- EC2 Instances: Linux servers running Apache web server
- Security Groups: Firewall rules controlling inbound and outbound traffic
- Auto Scaling Group: Automatically adjusts server capacity based on demand

2. Architecture Diagram

The following diagram illustrates the complete AWS infrastructure architecture for this project:

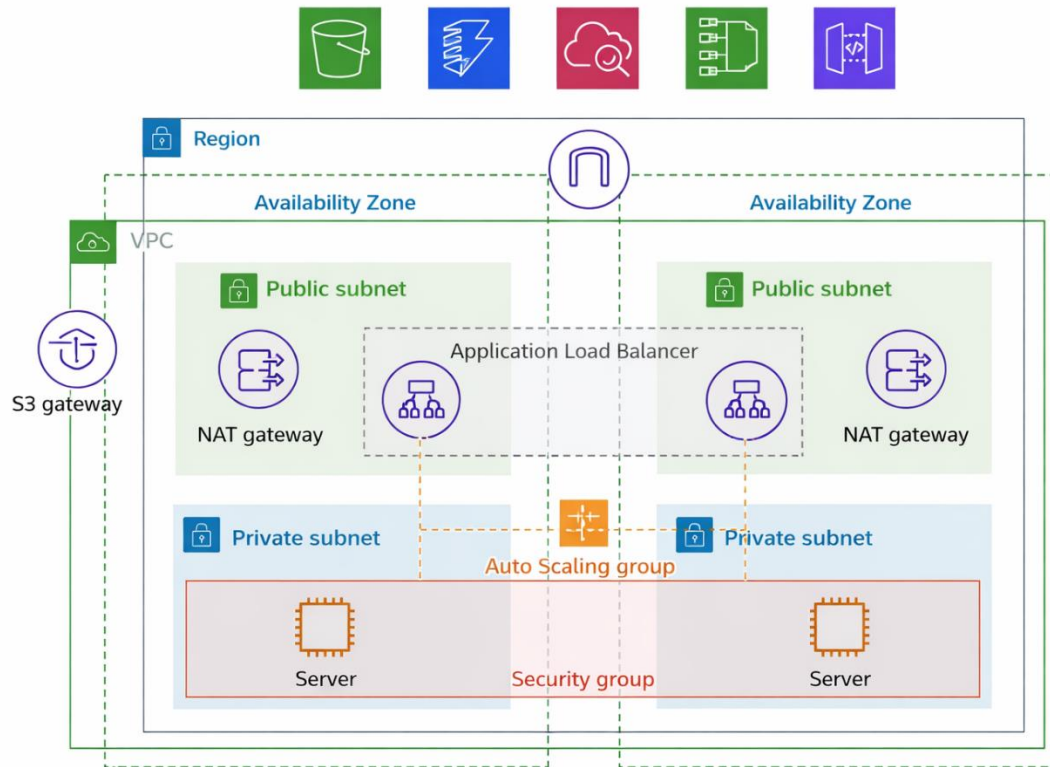


Figure 1: AWS VPC Architecture with High Availability Design

Architecture Components Explained

Region: The entire infrastructure is deployed within a single AWS region for optimal performance and compliance.

Availability Zones: Resources are distributed across two availability zones to ensure fault tolerance and high availability.

VPC (Virtual Private Cloud): A logically isolated network environment where all AWS resources are deployed.

Public Subnets: Subnets with direct internet access via Internet Gateway, hosting the Application Load Balancer and NAT Gateways.

Private Subnets: Subnets without direct internet access, hosting web servers for enhanced security.

NAT Gateway: Allows instances in private subnets to access the internet for updates while maintaining security.

Application Load Balancer: Distributes incoming HTTP/HTTPS traffic across multiple web servers for load distribution and fault tolerance.

Security Groups: Virtual firewalls controlling inbound and outbound traffic to EC2 instances.

3. Prerequisites

Before starting this project, ensure you have the following requirements in place:

AWS Account Requirements

- Active AWS account with appropriate permissions
- IAM user with EC2, VPC, and related service permissions
- Understanding of AWS Free Tier limits to avoid unexpected charges

Software Requirements

- PuTTY: SSH client for Windows (download from <https://www.putty.org/>)
- PuTTYgen: Key generation tool (included with PuTTY installer)
- Text editor: Notepad++, VS Code, or similar for editing configuration files
- Web browser: For accessing AWS Console and testing the deployed website

Knowledge Prerequisites

- Basic understanding of cloud computing concepts
- Familiarity with Linux command line
- Basic HTML knowledge
- Understanding of networking concepts (IP addresses, subnets, routing)

4. Step-by-Step Implementation

4.1 VPC and Network Setup

AWS provides a streamlined 'VPC and more' option that automatically creates all necessary networking components in one step, including VPC, subnets, route tables, Internet Gateway, and NAT Gateways.

Step 1: Create VPC Using 'VPC and More'

1. Log in to AWS Management Console
2. Navigate to VPC Dashboard
3. Click 'Create VPC'

4. Select 'VPC and more' option (this automates the entire setup)

The screenshot shows the AWS 'Create VPC' console. The 'VPC settings' section on the left includes:

- Resources to create:** Two radio buttons, 'VPC only' and 'VPC and more'. 'VPC and more' is selected.
- Name tag auto-generation:** A checkbox labeled 'Auto-generate' is checked. Below it is a text input field containing 'production'.
- IPv4 CIDR block:** A text input field contains '10.0.0.0/16'. To its right, it says '65,536 IPs'. Below the input, a note states: 'CIDR block size must be between /16 and /28.'
- IPv6 CIDR block:** Two radio buttons, 'No IPv6 CIDR block' (selected) and 'Amazon-provided IPv6 CIDR block'.
- Tenancy:** A dropdown menu set to 'Default'.

The 'Preview' section on the right shows a diagram of the network setup:

- VPC:** A box labeled 'production-vpc' with a link 'Show details'.
- Subnets (4):** A box labeled 'Subnets within this VPC' containing four subnets:
 - eu-west-3a:** Contains 'production-subnet-public1-eu-' (marked with a green 'A') and 'production-subnet-private1-eu-' (marked with a blue 'A').
 - eu-west-3b:** Contains 'production-subnet-public2-eu-' (marked with a green 'B') and 'production-subnet-private2-eu-' (marked with a blue 'B').
- Route table:** A box labeled 'Route network' containing three entries, each labeled 'production-'.

Lines connect the VPC box to the subnets, and the subnets to the route table.

Step 2: Configure VPC Settings

Basic Configuration:

- Name tag auto-generation: Enable (or enter custom name like 'WebServer-VPC')
- IPv4 CIDR block: 10.0.0.0/16 (provides 65,536 IP addresses)

Number of Availability Zones:

- Select '2' (for high availability across two AZs)

Number of public subnets:

- Select '2' (one in each AZ)

Number of private subnets:

- Select '2' (one in each AZ for web servers)

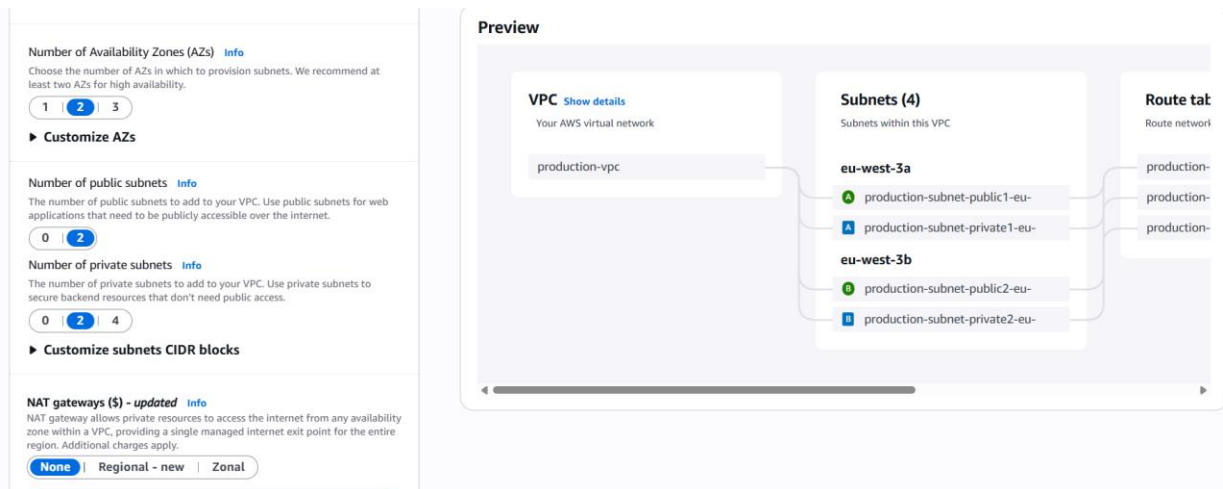
NAT gateways:

- Select '1 per AZ' (recommended for high availability)

Note: This creates 2 NAT Gateways (one in each AZ). You can select 'In 1 AZ' to save costs in development environments.

VPC endpoints:

- Select 'None' (can be added later if needed for S3 or other services)



Step 3: Review and Create

1. Review the preview diagram showing your VPC architecture

2. Verify the following will be created automatically:

- VPC with specified CIDR block
- 2 Public subnets (10.0.0.0/24 and 10.0.1.0/24)
- 2 Private subnets (10.0.128.0/24 and 10.0.129.0/24)
- Internet Gateway (automatically attached)
- 2 NAT Gateways (one per AZ in public subnets)
- Route tables (configured for public and private routing)
- Network ACLs (default allow all)

3. Click 'Create VPC'

4. Wait 2-5 minutes for all resources to be created (NAT Gateways take the longest)

VPC > Your VPCs

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

VPCs

VPC encryption controls

2

info

Find VPCs by attribute or tag

Last updated less than a minute ago

Actions

Create VPC

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv
☐	production-vpc-vpc	ypc-083d6be3a4b7740f5	Available	-	-	Off	10.0
☐	-	ypc-0b5a3d9d66bd264a6	Available	-	-	Off	172

Select a VPC above

4.2 EC2 Auto Scaling Configuration

Instead of manually launching individual EC2 instances, we'll use Auto Scaling Groups for automatic scaling and high availability. Auto Scaling automatically adjusts the number of instances based on demand and ensures instances are distributed across multiple Availability Zones.

Step 1: Create Security Group

1. Navigate to EC2 Dashboard → Security Groups
2. Click 'Create security group'
3. Configure:
 - Name: "XXXX"
 - Description: Security group for web servers
 - VPC: the one which you created above
4. Add Inbound Rules:
 - SSH (Port 22): Source: My IP (for secure access)
 - HTTP (Port 80): Source: 0.0.0.0/0 (public web access)
 - HTTPS (Port 443): Source: 0.0.0.0/0 (for future SSL)

Step 2: Create Launch Template

A launch template defines the instance configuration that Auto Scaling will use to launch new instances.

1. Go to EC2 Dashboard → Launch Templates
2. Click 'Create launch template'
3. Launch template name: WebServer-Launch-Template
4. Template version description: Initial version with Apache setup
5. Check 'Provide guidance to help me set up a template that I can use with EC2 Auto Scaling'

Application and OS Images (AMI):

- Select 'Amazon Linux 2023 AMI' (free tier eligible) or ubuntu , here I used ubuntu
- Architecture: 64-bit (x86)

Instance type:

- t2.micro (free tier eligible)

Key pair:

- Create new key pair or select existing
 - Name: webserver-keypair

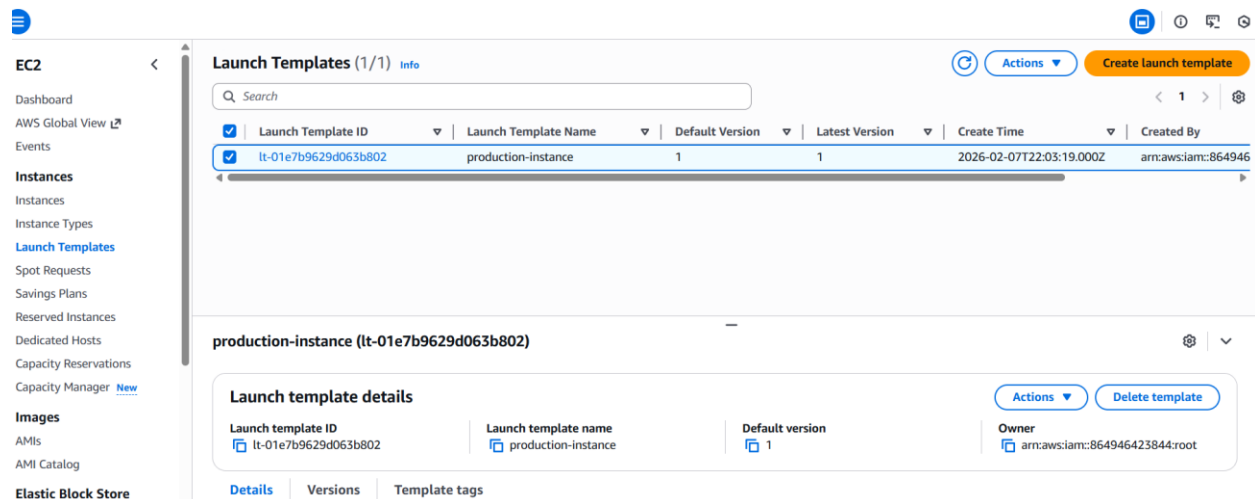
- Type: RSA
- Format: .pem (for Linux/Mac) or .ppk (for PuTTY on Windows)

Network settings:

- Don't include in launch template (will be specified in Auto Scaling Group)
- Security groups: Select WebServer-SG

Storage (volumes):

- 8 GiB gp3 (default is fine)



Step 3: Create Auto Scaling Group

1. Go to EC2 Dashboard → Auto Scaling Groups
2. Click 'Create Auto Scaling group'
3. Name: "xxxx"

Step 1: Choose launch template

- Select: WebServer-Launch-Template
- Click 'Next'

Step 2: Choose instance launch options

- VPC: Select your VPC (created with 'VPC and more')
- Availability Zones and subnets: Select BOTH private subnets
 - Private subnet | us-west-3a (or your first AZ)
 - Private subnet | us-west-3b (or your second AZ)
- Click 'Next'

Step 3: Configure advanced options

- Load balancing: Select 'Attach to a new load balancer' (if needed)
 - Load balancer type: Application Load Balancer
 - Load balancer name: WebServer-ALB
 - Load balancer scheme: Internet-facing
 - Availability Zones: Select both public subnets
- Health checks:
 - Turn on Elastic Load Balancing health checks
 - Health check grace period: 300 seconds
- Click 'Next'

Step 4: Configure group size and scaling

- **Desired capacity: 2**
- **Minimum capacity: 1**
- **Maximum capacity: 4**
- Scaling policies: Target tracking scaling policy (optional)
 - Metric type: Average CPU utilization
 - Target value: 50%
- Click 'Next'

Step 5: Add notifications (optional)

- Click 'Next' (skip for now)

Step 6: Add tags

- Key: Name
- Value: WebServer-ASG-Instance
- Click 'Next'

Step 7: Review

- Review all settings
- Click 'Create Auto Scaling group'

Auto Scaling will now automatically launch 2 instances (your desired capacity) and distribute them across the two private subnets in different Availability Zones for high availability.

EC2 > Auto Scaling groups

Auto Scaling groups (1/1) Info Last updated less than a minute ago [Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

☒	Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max
☒	production-instance	production-instance Version Default	2	-	2	1	4

Auto Scaling group: production-instance

[Details](#) [Integrations](#) [Automatic scaling](#) [Instance management](#) [Instance refresh](#) [Activity](#) [Monitoring](#) [Tags - moved](#)

production-instance Capacity overview [Edit](#)

 arn:aws:autoscaling:eu-west-3:864946423844:autoScalingGroup:88f27686-707e-42bb-b449-d4cc29d40b64:autoScalingGroupName/production-instance

Desired capacity	Scaling limits	Desired capacity type	Status
2	1 - 4	Units (number of instances)	-

Date created
Sat Feb 07 2026 23:04:48 GMT+0100 (Central European Standard Time)

Settings

Step 3: Convert PEM to PPK for PuTTY

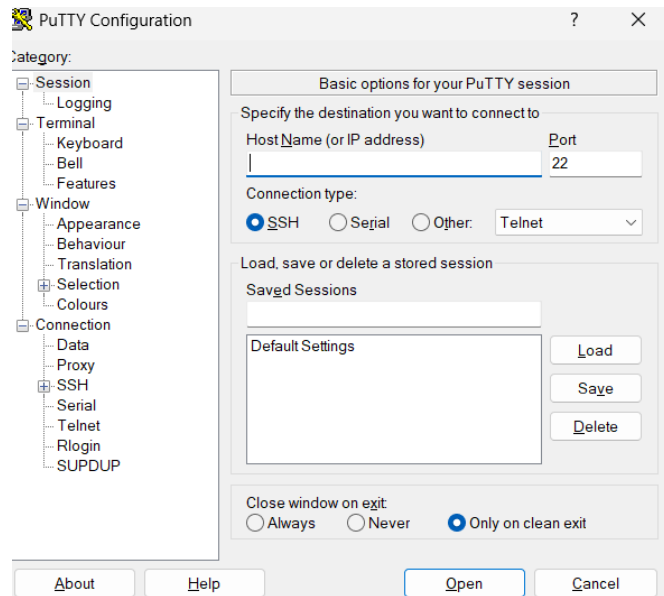
1. Open PuTTYgen application
2. Click 'Load' and select your .pem key file
3. Change file filter to 'All Files (*.*)' to see .pem files
4. Click 'Save private key'
5. Save as '<name>.ppk' (you'll use this with PuTTY)

4.3 SSH Connection using PuTTY

Connect to your EC2 instance using PuTTY to configure the Apache web server.

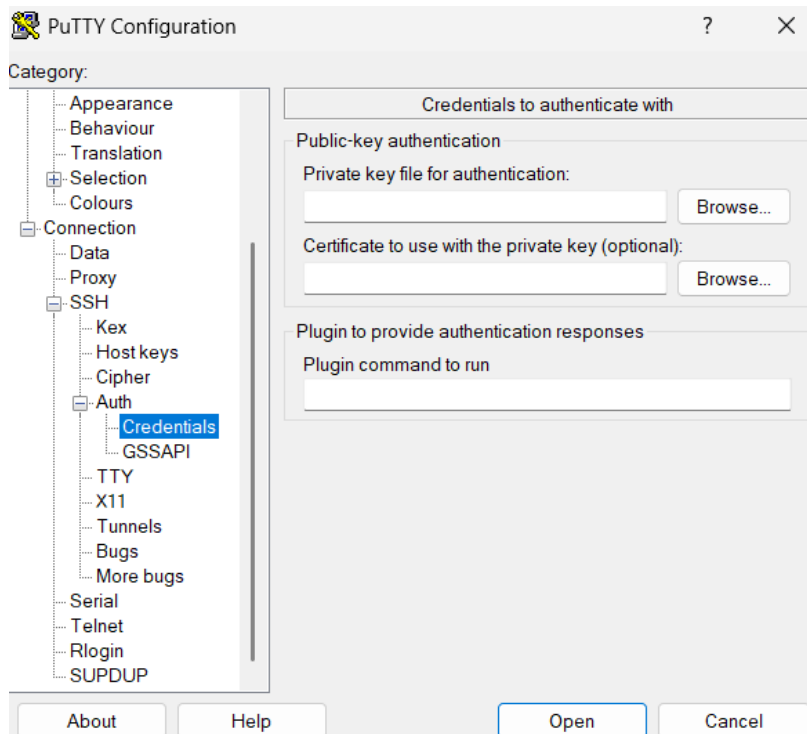
Step 1: Get Instance Details

1. Go to EC2 Dashboard → Instances
2. Select your instance (WebServer-1)
3. Copy the Private IPv4 address (e.g., 10.0.3.15)



Step 2: Configure PuTTY

1. Open PuTTY application
2. In the 'Session' category:
 - Host Name: ec2-user@<private-ip-address>
 - Port: 22
 - Connection type: SSH
3. Navigate to Connection → SSH → Auth → Credentials
4. Browse and select your .ppk file (webserver-keypair.ppk)
5. (Optional) Go back to Session, enter name 'WebServer' and click 'Save'
6. Click 'Open' to connect



Step 3: First Connection

1. PuTTY will show a security alert on first connection
2. Click 'Accept' to trust the host key
3. You should now see the Amazon Linux terminal prompt

4.4 Apache Installation and Configuration

Now that you're connected to the server via SSH, let's install and configure Apache web server.

Step 1: Update System Packages

Run the following command to update all system packages:

```
sudo apt update
```

This ensures all packages are up-to-date before installing Apache.

Step 2: Install Apache Web Server

Install Apache HTTP Server (httpd) using the following command:

```
sudo apt install -y apache2
```

Wait for the installation to complete. You should see a 'Complete!' message when finished. The '-y' flag automatically confirms all prompts.

Step 3: Start Apache Service

Start the Apache web server:

```
sudo systemctl start apache2
```

Step 4: Enable Apache to Start on Boot

Configure Apache to automatically start when the server boots:

```
sudo systemctl enable apache2
```

Step 5: Verify Apache Status

Check that Apache is running properly:

```
sudo systemctl status httpd
```

You should see output showing that httpd is 'active (running)'. Press 'q' to exit the status view.

4.5 HTML Deployment

Deploy custom HTML content to your web server.

Step 1: Navigate to Web Root Directory

Apache serves files from `/var/www/html` by default. Navigate to this directory:

```
cd /var/www/html
```

Step 2: Create HTML File

Create a new `index.html` file using a text editor:

```
sudo nano /var/www/html/index.html
```

Step 3: Add HTML Content

Enter your HTML code. Here's a sample template:

Instance 1 ;

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Welcome to My EC2</title>
```

```
<style>
```

```
body {
```

```
    margin: 0;
```

```
    height: 100vh;
```

```
    display: flex;
```

```
    justify-content: center;
```

```
    align-items: center;
```

```
    background: linear-gradient(135deg, #0f2027, #203a43, #2c5364);
```

```
    font-family: Arial, Helvetica, sans-serif;
```

```
    color: white;
```

```
}
```

```
.card {
```



```
    background: rgba(255, 255, 255, 0.15);
    padding: 50px 70px;
    border-radius: 18px;
    box-shadow: 0 20px 50px rgba(0,0,0,0.4);
    text-align: center;
}

h1 {
    margin-bottom: 10px;
    font-size: 38px;
}

p {
    font-size: 18px;
    margin: 6px 0;
    opacity: 0.9;
}

.badge {
    margin-top: 18px;
    display: inline-block;
    padding: 10px 22px;
    background: #00e676;
    color: #0b3d2e;
    border-radius: 999px;
    font-weight: bold;
}

</style>
</head>
```

```
<body>
  <div class="card">
    <h1><img alt="EC2 logo" data-bbox="178 158 198 178"/> Welcome to EC2</h1>
    <p>Apache Web Server is running</p>
    <p>Served from this instance</p>
    <div class="badge">Healthy & Online</div>
  </div>
</body>
</html></html>
```

Instance 2 ;

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>EC2 Instance C</title>
```

```
  <style>
```

```
    body {
```

```
      margin: 0;
```

```
      height: 100vh;
```

```
      display: flex;
```

```
      justify-content: center;
```

```
      align-items: center;
```

```
      background: radial-gradient(circle at top, #232526, #414345);
```

```
      font-family: "Segoe UI", Tahoma, sans-serif;
```

```
      color: #ffffff;
```

```
    }
```

```
.card {  
  background: rgba(0, 0, 0, 0.45);  
  padding: 50px 70px;  
  border-radius: 20px;  
  box-shadow: 0 25px 60px rgba(0,0,0,0.6);  
  text-align: center;  
  border: 1px solid rgba(255,255,255,0.15);  
}  
  
h1 {  
  margin: 0 0 12px;  
  font-size: 40px;  
  letter-spacing: 1px;  
}  
  
p {  
  margin: 6px 0;  
  font-size: 18px;  
  opacity: 0.9;  
}  
  
.badge {  
  margin-top: 20px;  
  display: inline-block;  
  padding: 10px 26px;  
  background: linear-gradient(135deg, #ff9800, #ff5722);  
  color: #1a1a1a;  
  border-radius: 999px;  
  font-weight: bold;
```

```
font-size: 14px;
text-transform: uppercase;
letter-spacing: 1px;
}
</style>
</head>
<body>
  <div class="card">
    <h1>⚡ EC2 Instance C</h1>
    <p>Running on Apache</p>
    <p>Behind Application Load Balancer</p>
    <div class="badge">Live & Serving</div>
  </div>
</body>
</html>
```

Step 4: Save the File

If using nano editor:

1. Press Ctrl + X to exit
2. Press Y to confirm save
3. Press Enter to confirm filename

4.6 Testing and Verification

Verify that your web server is working correctly and accessible.

Step 1: Test Locally on Server

Test Apache from within the EC2 instance:

```
curl http://localhost
```

You should see your HTML code output in the terminal. This confirms Apache is serving your page.

Step 2: Get Public IP or Load Balancer DNS

If using Application Load Balancer:

1. Go to EC2 Dashboard → Load Balancers
2. Select your load balancer
3. Copy the DNS name (e.g., my-alb-123456789.us-east-1.elb.amazonaws.com)

If using direct EC2 access (instance in public subnet):

1. Go to EC2 Dashboard → Instances
2. Select your instance
3. Copy the Public IPv4 address

Step 3: Access from Web Browser

1. Open your web browser
2. Enter the URL:
 - For Load Balancer: `http://<load-balancer-dns>`
 - For Public IP: `http://<public-ip-address>`
3. You should see your custom HTML page displayed

Step 4: Verify Security Group Rules

If you cannot access the page, verify:

- Security group allows HTTP (port 80) from 0.0.0.0/0
- Instance is in a public subnet (if accessing directly)
- Load balancer target group health checks are passing
- Apache service is running (check with `sudo systemctl status httpd`)

5. Security Configuration

Implement security best practices to protect your web server infrastructure.

Security Group Best Practices

Principle of Least Privilege:

- Only allow necessary ports and protocols
- Restrict SSH access to specific IP addresses (your IP or office IP range)
- Regularly audit and update security group rules

Key Pair Management

- Store private keys (.pem and .ppk files) securely
- Never share private keys or commit them to version control
- Use separate key pairs for different environments (dev, staging, production)
- Rotate keys regularly (every 90 days recommended)

System Updates and Patching

Regular system updates are critical for security:

Weekly update command

```
sudo yum update -y
```

Network ACLs and Private Subnets

- Keep application servers in private subnets when possible
- Use NAT Gateways for outbound internet access from private subnets
- Implement Network ACLs as an additional layer of security
- Never expose databases or backend services directly to the internet

SSL/TLS Encryption (Future Enhancement)

For production deployments, implement HTTPS:

- Obtain SSL certificate from AWS Certificate Manager (ACM)
- Configure Application Load Balancer to terminate SSL
- Redirect HTTP traffic to HTTPS
- Enable HTTP Strict Transport Security (HSTS)

6. Troubleshooting

Common issues and their solutions.

Cannot Connect via PuTTY

Problem: Connection timeout or 'Network error: Connection refused'

Solutions:

1. Verify security group allows SSH (port 22) from your IP
2. Check if using correct IP address (public for public subnet, private if using bastion)
3. Ensure instance is in 'running' state
4. Verify correct username (ec2-user for Amazon Linux)
5. Confirm you're using the correct .ppk file

Website Not Accessible

Problem: Cannot access website from browser

Solutions:

1. Check if Apache is running: `sudo systemctl status httpd`
2. Verify security group allows HTTP (port 80) from 0.0.0.0/0
3. Ensure using `http://` not `https://` (unless SSL configured)
4. Verify load balancer target group health is 'healthy'
5. Check Apache error logs: `sudo tail -f /var/log/httpd/error_log`

Apache Won't Start

Problem: Apache fails to start or stops unexpectedly

Solutions:

1. Check error logs: `sudo journalctl -u httpd -n 50`
2. Verify configuration syntax: `sudo apachectl configtest`
3. Check if port 80 is already in use: `sudo netstat -tulpn | grep :80`
4. Ensure sufficient disk space: `df -h`

Page Shows Default Apache Test Page

Problem: Seeing default Apache page instead of your HTML

Solutions:

1. Verify index.html exists: `ls -la /var/www/html/`
2. Check file permissions: should be 644
3. Ensure file is named exactly 'index.html' (case-sensitive)
4. Clear browser cache and refresh

Useful Commands Reference

Check Apache status

```
sudo systemctl status httpd
```

View Apache error logs

```
sudo tail -f /var/log/httpd/error_log
```

View Apache access logs

```
sudo tail -f /var/log/httpd/access_log
```

Restart Apache

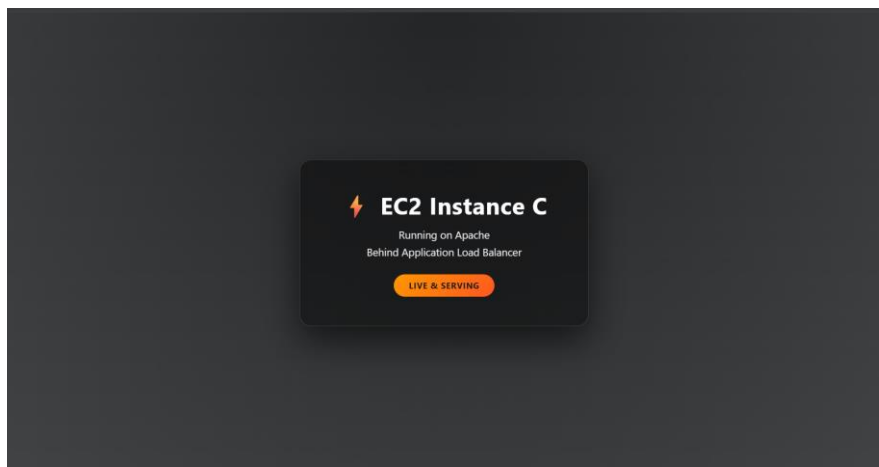
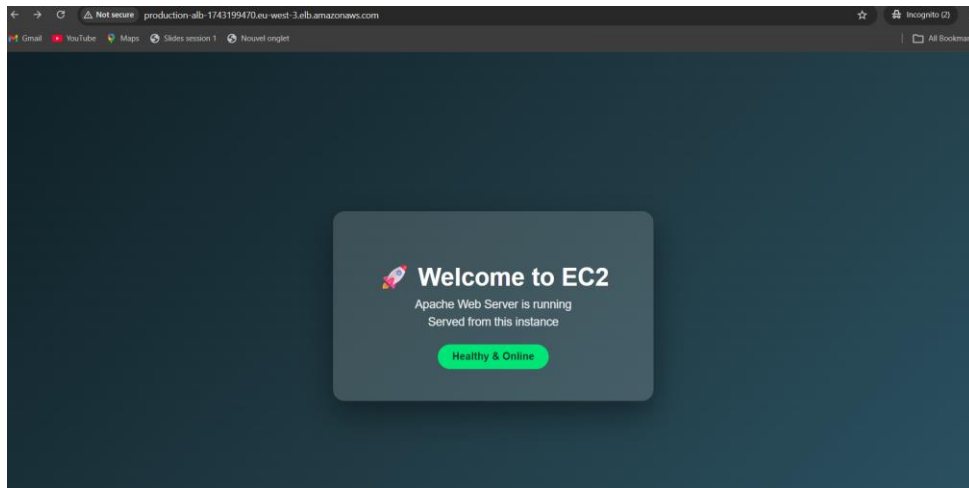
```
sudo systemctl restart httpd
```

Test Apache configuration

```
sudo apachectl configtest
```

Check listening ports

```
sudo netstat -tulpn | grep httpd
```



Conclusion

Congratulations! You have successfully deployed a project used in production on AWS with a highly available architecture. This project demonstrates fundamental cloud computing skills including VPC networking, EC2 instance management, load balancing, and web server administration.

Next Steps

- Configure Auto Scaling to automatically adjust capacity based on traffic
- Set up CloudWatch monitoring and alarms for proactive management
- Implement a database tier using Amazon RDS
- Add a CDN using Amazon CloudFront for improved global performance
- Implement backup strategies using AWS Backup or EBS snapshots
- Explore Infrastructure as Code using AWS CloudFormation or Terraform

Remember to stop or terminate resources when not in use to avoid unnecessary charges.

End of Documentation